

Conversation Context Protocol

Zotero AI Sidebar

2026-05-14

Contents

Conversation Context Protocol

1

TL;DR

1

1. 单轮请求的实际形状

2

2. 三层上下文机制

2

3. Codex CLI 的同类机制

3

4. 对齐点 vs 分歧点

4

5. 可调旋钮 (src/context/policy.ts)

4

6. 开放问题 / 未来工作

5

参考文件索引

5

Conversation Context Protocol

本文档回答一个具体问题：**这个插件每次调用大模型 API 时，到底把哪些“历史”塞进了请求里？什么是模型端控制的，什么是本地控制的？**以及这套设计与 OpenAI Codex CLI 的同类机制如何对照。

读完应能：- 在不打开调试器的情况下推断出任意一轮 wire payload 的形状；- 知道改哪个常量可以让“历史回放”更激进或更克制；- 看懂 message-format.ts 里 toApiMessages 与 formatContextLedger 为什么是两条独立路径。

TL;DR

数据	是否每轮都发	控制位置
完整 user/assistant 对话文本	是， 全发	sidebar.ts 直接拼 [...history, userMessage]
历史轮的 function_call / function_call_output 对	是，全发	openai.ts 在 tool loop 内累加 input 数组
历史轮 新附带 的 PDF 选区 / 标注 / 检索片段	看预算（最近 4 轮 + 8000 字共享额度）	message-format.ts retainedRecentContextIndexes
历史轮的 PDF 全文	从不重发 (INVARIANT)	message-format.ts 头部注释 + 无对应代码路径
历史上下文摘要账本 (ledger)	是， 拼到 system prompt 末尾	sidebar.ts contextAwareSystemPrompt
服务器端记忆 (previous_response_id)	不使用	openai.ts 显式 store: false

模型端**没有**任何会话状态。“何时复用上下文”完全由本地代码决定，模型只能基于当前请求里看到的東西作答；如需重新看历史 PDF 段落，必须自己调 chat_get_previous_context 工具。

1. 单轮请求的实际形状

入口：src/modules/sidebar.ts:4033

```
const messagesForApi: Message[] = toApiMessages([
  ...history, userMessage],
  { message: userMessage },
  contextPolicy,
);
```

- history = state.messages.slice() (sidebar.ts:3657)，即当前 Zotero item 的整段聊天 thread。
- userMessage 是本轮新输入。
- toApiMessages 不裁剪历史消息数量；它只决定**哪些历史轮的 context 块要被重新内联到那条 user message 的 content 里**。

接着在 src/providers/openai.ts:150-159（无工具时）或 :319-336（带工具循环时），整段 input 连同 system prompt 一起发出去：

```
client.responses.create({
  model: preset.model,
  instructions: systemPrompt,      // 拼了 ledger 的 system 文本
  input: toOpenAIInput(messages),  // 整段 thread + 历史 tool 调用
  store: false,                    // INVARIANT: 不依赖服务器状态
  ...
});
```

store: false 的代价就是 previous_response_id 链路被废掉——服务器不留 response item，每轮都得自己把状态送过去。这是有意为之的：见 openai.ts:21-38 的注释。

2. 三层上下文机制

src/context/message-format.ts 把“历史”拆成三种东西，各自用不同强度回放。

第一层：对话文本 — 全量回放

toApiMessages 默认逐条 map，每条历史消息的 content 原样进 payload。这一层没有 token 预算控制，长 thread 会单调增长。**当前没有 compaction 实现**——见后文“开放问题”。

第二层：历史 context 块 — 滑窗 + 字数预算 + 签名去重

retainedRecentContextIndexes (message-format.ts:452-476) 三个守卫按顺序跑：

1. **轮次窗口**：只看最近 policy.retainedContextTurnCount = 4 轮 user 消息；
2. **字数预算**：所有被回放的轮次共享 policy.retainedContextCharBudget = 8000 字，先到先得；
3. **签名去重**：相同选区文本/相同 PDF 字符范围/相同标注集合，只保留更近的一轮。原因写在 message-format.ts:444-451：用户反复问同一段落时不要重复 3 次发送，也避免 cache-bust Anthropic 的 ephemeral cache。

被选中的历史轮，其 context 块 ([Selected PDF text] / [Zotero annotations] / [Retrieved PDF passages]) 会重新嵌入那条历史 user message 的 content 里——在 wire 上看不出“这段是历史回放”，模型读到的就像那条消息原本就带着这段内容。

第三层：上下文账本 (ledger) — 全量元数据，零原文

formatContextLedger (message-format.ts:235-318) 扫每条历史 user message 的 context，输出每行一段紧凑文本：

```
- turn 3; mode=search_pdf; source_id="..."; pdf_passages=2; pdf_passage_chars=1840; pdf_ranges=4500-5800,7100-7300; previous_context_tool=chat_get_previous_context
```

- turn 5; mode=full_pdf; full_pdf_chars=92000; full_pdf_truncated=true

这串元数据被 contextAwareSystemPrompt (sidebar.ts:4292-4296) 拼到 system prompt 末尾，并附带一段使用说明：模型可以把它当作“我已经看过什么”的地图，需要重读历史片段时自己调 chat_get_previous_context 工具——而不是本地代码替它判断。

INVARIANT: 原文 PDF 全文/远程论文文本**从不**进 ledger，也不进任何回放路径。它们只在 message 的 context 元数据里留一个字数标记。message-format.ts:15-17 把这一条写死了。

3. Codex CLI 的同类机制

引用基于 openai/codex 仓库 main 分支 (commit 6a225e4)，文件路径全部相对 codex-rs/。

跨轮上下文 — 也是全量重发

请求装配在 core/src/client.rs::build_responses_request，关键字段：

- input: prompt.get_formatted_input() — get_formatted_input() 在 core/src/client_common.rs 里克隆 Prompt.input: Vec<ResponseItem> 整段历史；
- store: provider.is_azure_responses_endpoint() — 即非 Azure 走 store: false，与本插件一致；
- instructions: &prompt.base_instructions.text — **只放基础 system 文本**。AGENTS.md、用户指令、环境上下文不拼进 system，而是作为 ResponseItem 注入 input 数组（见 core/src/context/contextual_user_message.rs、core/src/context/mod.rs）。

Turn loop — needs_follow_up

core/src/session/turn.rs::run_turn 与 try_run_sampling_request 内的 needs_follow_up 标志驱动多轮重采样。每次再采样前都会重建请求：

```
sess.clone_history().await.for_prompt(&model_info.input_modalities)
```

也就是说每个 sub-turn 都把整段 ContextManager（含 tool 调用与输出）重新序列化进新的 input。本插件 openai.ts:319-465 的 for (let iteration = 0; ...) 循环是它的 TS 镜像版本——同样累加 input、同样 parallel_tool_calls: false、同样把 function_call 与 function_call_output 配对放进下一轮请求。

差别仅在于 Codex 有“是否需要继续采样”这个语义信号，而这里用 maxToolIterations 安全熔断 + “本轮无新 tool call ⇒ 退出循环”的硬规则。两者都不基于“用户意图类型”决定迭代数（openai.ts:36-38 写明这是**安全熔断不是路由逻辑**，对应项目 CLAUDE.md 的非协商项）。

Compaction — Codex 有，这里没有

core/src/compact.rs + core/src/tasks/compact.rs: - 触发: token 压力下 run_inline_auto_compact_task() 自动触发；用户 /compact 手动触发；- 策略: 调模型生成摘要，replace_compacted_history() 在内存里**就地替换** history。assistant 轮被丢弃，最近 user 消息保留至 COMPACT_USER_MESSAGE_MAX_TOKENS = 20_000；- 摘要载体: 摘要被包成一条 **user-role ResponseItem::Message** (前缀来自 core/src/templates/compact/summary_prefix.md)，重新插回 input 数组——而不是塞进 instructions。

持久化 — Rollouts (JSONL)

core/src/rollout.rs::RolloutRecorder 写 JSONL；schema 见 protocol/src/protocol.rs::RolloutItem:

```
enum RolloutItem { SessionMeta, ResponseItem, Compacted, TurnContext, EventMsg }
```

恢复路径 (core/src/session/rollout_reconstruction.rs::reconstruct_history_from_rollout) 反向扫到最近的 Compacted{replacement_history} 检查点，然后正向重放。**持久化形态 ≈ wire 形态**: 磁盘上躺着的 ResponseItem 几乎可以直接送回 API。

本插件 src/settings/chat-history.ts 持久化的是富域 Message 对象（含 context、thinking、images、annotationDraft、task），**不是** wire 形态。要还原 wire 形态必须走 toApiMessages + formatContextLedger 的转换。两种取舍：Codex 的 wire-shape 持久化重放更便宜，本插件的富域持久化 UI 重渲染更便宜。

4. 对齐点 vs 分歧点

对齐

维度	Codex	本插件
标准 OpenAI 端点是否使用 server-side state	否 (store: false)	否 (store: false)
是否每轮重发完整 history	是	是
function_call / function_call_output 是否本地保存并重放	是	是
多轮 tool loop 是否避免 “基于用户意图分类决定迭代数”	是 (needs_follow_up 是机械信号)	是 (maxToolIterations 是安全熔断)
parallel_tool_calls	false	false (openai.ts:331)

分歧

核心分歧：历史摘要的注入位置。

- **Codex：**摘要 = 一条 user-role ResponseItem，注入 input。compaction 之后旧 ResponseItem 在 history 里被物理替换，只剩摘要那条。instructions 始终稳定（利于 prompt cache）。
- **本插件：**ledger = system prompt 后缀 (sidebar.ts:4292-4296)。每轮历史一变，system prompt 文本就跟着变；旧的 Message[] 在 state.messages 里完整保留，只是回放时按预算筛选。

这个分歧的实际影响：1. **Prompt cache 命中率：**Anthropic 的 ephemeral cache 与 OpenAI 的自动 cache 都对 system prompt 的 “前缀稳定性” 敏感。ledger 每轮变 ⇒ system prompt 后缀变 ⇒ 系统提示词的 cache 命中只能覆盖 ledger 之前的部分。Codex 的做法把变动放在 input 末尾，instructions 不动，cache 命中范围更大。2. **可读性：**本插件的 ledger 让人类用户在 Markdown 导出里能看到 “模型当时被告告诉了什么历史摘要”，调试更直观。Codex 把摘要混在 user 消息流里，对自动化重放更友好但对人工审计更隐晦。3. **Compaction 行为：**本插件目前**没有** compaction——长 thread 单调增长直到上下文窗口炸掉。Codex 的 replace_compacted_history() 是真删旧消息。

次要分歧：

- **持久化 schema：**Codex RolloutItem 接近 wire shape；本插件 Message 是富域 UI shape。
- **previous_response_id：**Codex 在 client.rs::prepare_websocket_request 里用，但**仅限 WebSocket 增量回放**，不是跨轮状态。本插件完全不用。两者跨轮行为一致。
- **历史标注/选区回放：**Codex 没有 “选区/检索片段” 这种结构化历史 context（它面对的是代码，不是 PDF）。本插件第二层 (retainedRecentContextIndexes) 是 Codex 没有对应物的领域专属机制。

5. 可调旋钮 (src/context/policy.ts)

常量	默认	改大的影响	改小的影响
retainedContextTurnCount	4	更老的轮次也能回放 context 块	只回放最近一两轮
retainedContextCharBudget	8000	单轮可塞更多回放字符	更激进地走 “只发账本，让模型自己 reload”
maxToolIterations	100	安全熔断更宽松（仅熔断用）	工具循环更早强制中止
fullPdfTokenBudget	60_000	zotero_get_full_pdf 单次返回更大	更小的全文截断
maxPassageChars / passageOverlapChars	1200 / 160	检索片段更长更连续	片段更细粒度但单段信息更碎

两个常见调整动机： - 用户抱怨 “模型老忘记上一轮我选了什么” → 调大 `retainedContextCharBudget` (保留更多回放)； - 用户抱怨 token 成本 → 调小到 0，**强制依赖 ledger + `chat_get_previous_context`**——但需先评估当前模型是否稳定地会主动调那个工具。

6. 开放问题 / 未来工作

1. **没有 compaction。**本插件 thread 单调增长。可选方向：
 - 学 Codex 实现真正的就地 compaction (需要新工具：让模型生成摘要、本地原子替换 `state.messages` 头段)；
 - 或者先实现 “超过阈值后只保留最近 N 轮 + 全量 ledger” 的硬截断(近 Codex 的 `COMPACT_USER_MESSAGE_MAX_TOKENS` 思路)。
 - 决策依据：用户实际使用中 thread 平均长度 vs 目标模型上下文窗口。
2. **Ledger 注入位置是否值得迁到 input。**如果开始追求 prompt cache 命中率 (尤其是 Anthropic provider 走长会话)，考虑把 ledger 改造成一条 user-role Message 注入 `messagesForApi` 末尾、instructions 保持稳定。需要在 UI 渲染端区分 “这是 ledger 不是真正的 user 输入”。
3. **持久化 schema 与 wire shape 的距离。**当前 Message 富域形态对应 UI 渲染需求，但每次重放都要走 `toApiMessages` 转换。如果未来要支持 “导出/导入 rollout 用于 replay 测试”，可能需要一个中间 wire-shape 的快照格式 (参考 Codex `RolloutItem`)。

参考文件索引

本仓库： - `src/modules/sidebar.ts:3657, 4033, 4292` — history 取片、`toApiMessages` 调用、ledger 拼接 - `src/context/message-format.ts:21-53, 235-318, 452-476` — wire 装配、ledger 格式、滑窗保留 - `src/context/policy.ts` — 全部上下文/工具预算 - `src/providers/openai.ts:21-38, 114-178, 298-475` — 设计 INVARIANT 注释、单次请求路径、tool loop 路径 - `src/settings/chat-history.ts` — 持久化层 (富域 Message schema) - `docs/HARNESS_ENGINEERING.md` — 上层 harness 设计契约

openai/codex (main @ 6a225e4)： - `codex-rs/core/src/client.rs::build_responses_request` — Responses API 请求装配 - `codex-rs/core/src/client_common.rs::Prompt::get_formatted_input` — input 重建 - `codex-rs/core/src/session/turn.rs::run_turn, try_run_sampling_request, needs_follow_up` — 多轮采样循环 - `codex-rs/core/src/compact.rs, codex-rs/core/src/tasks/compact.rs` — compaction 实现 - `codex-rs/core/src/rollout.rs::RolloutRecorder` + `protocol/src/protocol.rs::RolloutItem` — 持久化 - `codex-rs/core/src/session/rollout_reconstruction.rs::reconstruct_history_from_rollout` — 恢复算法 - `codex-rs/core/src/templates/compact/summary_prefix.md` — 摘要消息前缀模板